Build Task — Project 1: Beginner LLM App — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — What you're building, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — What you're building, and the exact pieces

Basic Version

You're combining everything from the 6 practice exercises above into one small chatbot: memory (so it remembers earlier turns), streaming (so replies feel instant), and safe handling of the failure types you already saw (bad key, too-long input, a structured-output request that doesn't come back clean).

Think of it as 3 files working together: one that talks to the model (`chat_client.py`), one that describes the shape of any structured answer you want back (`schemas.py`), and one that runs the actual terminal loop (`main.py`).

Here are the exact pieces you need to look up and use:

- `load_config()` and `get_logger()` from Doc01 — call them at the very top of `main.py` .
- The multi-turn `history` list pattern from the conversation-memory exercise.
- The streaming loop from the streaming-replies exercise.
- The `ExtractedInfo` model and the `refusal` check from the structured-output exercise.
- `try/except` blocks for `openai.AuthenticationError` and `openai.BadRequestError` , one each — not one shared catch-all.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

The Build Task is a synthesis, not new material — every function here is a small variation on something you already built in the 6 exercises above. `chat_client.py` should expose `create_client()`, `send_message(history, user_input) -> str`, and `stream_message(history, user_input)`, so `main.py` never touches the OpenAI SDK directly — it only calls your own, already-tested wrapper functions.

Reusing Doc01's `config.py` / `logging_setup.py` here isn't optional decoration — it's the whole point of building them in Doc01: this is the first of many projects that starts by importing them unchanged.

- `from config import load_config` / `from logging_setup import get_logger` — same functions from Doc01, unchanged, called at the top of `main.py`.
- `history: list[dict]` — the growing message list; every user turn and assistant reply gets appended to it, in order.
- Streaming: reuse `stream_message()`'s loop, but have it also return the accumulated `full_reply` string so `main.py` can append it to `history`.
- `schemas.py`: the `ExtractedInfo` model from the structured-output exercise (`name` and `intent`, both `str | None`). `get_structured_reply()` is that exercise's `extract_info()`: `client.beta.chat.completions.parse(response_format=ExtractedInfo, ...)`, check `refusal`, and return `None` when nothing useful came back.
- Separate `except` blocks matching the "Handles gracefully" requirements: `openai.AuthenticationError`, `openai.BadRequestError` (context length), and a Pydantic `ValidationError` plus the `refusal` check for a structured-output reply that didn't come back clean.

The one genuinely new design question the Build Task raises on its own: **what should happen to `history` after a `BadRequestError`?** Losing the whole conversation and starting over is safe but harsh. Silently dropping the oldest turns and continuing (as sketched below) keeps the conversation alive, at the cost of the model "forgetting" the earliest parts — a real trade-off, not a free fix.

```
except openai.BadRequestError:
    logger.warning("Conversation exceeded context length, trimming history.")
    print("That got too long — forgetting the oldest messages and continuing.")
    history = [history[0]] + history[-4:]
```

Sketch the rest of `main.py`'s loop — the streaming call, the structured-output branch, and both `except` blocks — before checking Hint 2.

Difference between Basic and Intermediate: Basic describes the pieces in plain words. Intermediate wires them into typed `chat_client.py` / `schemas.py` / `main.py` files and makes the one Build-Task-specific decision none of the individual exercises could make on their own — what to do with `history` the moment it goes over the limit.

Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
main.py:
    load config and logger from Doc01

    make the chat client
    start history with a system message

    loop forever:
        get user input
        try:
            stream the reply, show it as it arrives
            add both turns to history
        except bad key:
            print a clean message, stop
        except too long:
            print a clean message, drop the oldest turns, keep going

    (separately, on request) call the structured-output mode:
        try:
            get a checked object back
        except it doesn't validate:
            print a clean "couldn't understand that" message
```

The trickiest part — the structured-output call and its own specific error handling:

```
from pydantic import BaseModel, ValidationError

class ExtractedInfo(BaseModel):
    name: str | None
    intent: str | None

def get_structured_reply(client, user_input):
    try:
        completion = client.beta.chat.completions.parse(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": user_input}],
            response_format=ExtractedInfo,
```

```
        )
    except ValidationError:
        return None
    message = completion.choices[0].message
    if message.refusal:
        return None
    info = message.parsed
    if info.name is None and info.intent is None:
        # caller should print a clean "couldn't understand that" message
        return None
    return info
```

Expected output if you run just this (nothing calls it yet): nothing — try wiring the rest of `main.py`'s loop yourself before checking the Solution.

Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

```
chat_client.py:
    def create_client() -> OpenAI: ...
    def send_message(client, history, user_input) -> str: ...   # non-streaming
    # prints as it streams, returns the full text once finished
    def stream_message(client, history, user_input) -> str: ...
    def get_structured_reply(client, user_input) -> ExtractedInfo | None: ...

schemas.py:
    class ExtractedInfo(BaseModel): ...

main.py:
    config = load_config()
    logger = get_logger(__name__)
    client = create_client()
    history = [{"role": "system", "content": "..."}]

    loop:
        user_input = input("You: ")
        try:
            stream_message(client, history, user_input)
        except openai.AuthenticationError:
            logger.error(...); print clean message; stop
        except openai.BadRequestError:
            print clean message; history = [history[0]] + history[-4:]
```

```
from pydantic import BaseModel, ValidationError
from openai import OpenAI

EXTRACT_PROMPT = (
    "Pull the person's name and what they want out of the message. "
    "If there is no name or no request, use null — never guess."
)


class ExtractedInfo(BaseModel):
    name: str | None
    intent: str | None
```

```
def get_structured_reply(
    client: OpenAI, user_input: str
) -> ExtractedInfo | None:
    try:
        completion = client.beta.chat.completions.parse(
            model="gpt-4o-mini",
            messages=[
                {"role": "system", "content": EXTRACT_PROMPT},
                {"role": "user", "content": user_input},
            ],
            response_format=ExtractedInfo,
        )
    except ValidationError:
        return None
    message = completion.choices[0].message
    if message.refusal:
        return None
    info = message.parsed
    if info.name is None and info.intent is None:
        return None
    return info
```

Wire this into `main.py` behind its own command (e.g. the user types `/extract ...`), with its own `if result is None:` branch printing a clean failure message. Try finishing the rest yourself, then compare against the Solution.

Difference between Basic and Intermediate: Basic's pseudocode and near-complete code prove the structured-output piece works in isolation. Intermediate wires that same piece into a typed `chat_client.py` / `main.py` structure with the 2 required `except` blocks.

Back to the Build Task · Hint 1 · Hint 2 · Solution

Solution

Read both depths — they're not "wrong, right," they're 2 real, valid stages of building the same app, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# chat_client.py
from openai import OpenAI

def create_client():
    return OpenAI()

def stream_message(client, history, user_input):
    history.append({"role": "user", "content": user_input})
    stream = client.chat.completions.create(
        model="gpt-4o-mini", messages=history, stream=True
    )
    full_reply = ""
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="")
            full_reply += piece
    print()
    history.append({"role": "assistant", "content": full_reply})
    return full_reply
```

```
# main.py
import openai
from config import load_config
from logging_setup import get_logger
from chat_client import create_client, stream_message

config = load_config()
logger = get_logger("main")
client = create_client()
```

```
history = [{"role": "system", "content": "You are a helpful assistant."}]

while True:
    user_input = input("You: ")
    try:
        stream_message(client, history, user_input)
    except openai.AuthenticationError:
        print("Login failed — check your API key.")
        break
    except openai.BadRequestError:
        print(
            "That conversation got too long — forgetting the oldest messages."
        )
        history = [history[0]] + history[-4:]
```

Expected output on a normal run: each reply streams to the screen, and the conversation remembers earlier turns.

This version works correctly and meets every Build Task requirement. It's missing type hints and the structured-output feature — both worth adding next.

Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — everything in `main.py`'s loop, minimal structure

Story — `chat_client.py`: `main.py` shouldn't call the OpenAI SDK directly — it should call your own, already-tested wrapper functions instead. This is the same `stream_message()` shape you already built and understood in the streaming-replies exercise, now the one version every later document/project imports. **If not:** every file in this app (and later, Doc05/Doc06's rewrites of it) would each build their own slightly-different streaming loop, instead of sharing one tested version.

```
# chat_client.py
from openai import OpenAI
from pydantic import ValidationError

from schemas import ExtractedInfo

# why: the same rule as the structured-output exercise — use null when
# a name or a request is missing, so the model never invents one
EXTRACT_PROMPT = (
    "Pull the person's name and what they want out of the message. "
    "If there is no name or no request, use null — never guess."
)


def create_client() -> OpenAI:
    return OpenAI()


def send_message(client: OpenAI, history: list[dict], user_input: str) -> str:
    # why: a plain, non-streaming call — useful for tests, and for any
    # caller that doesn't need text to appear piece by piece.
    history.append({"role": "user", "content": user_input})
    response = client.chat.completions.create(
        model="gpt-4o-mini", messages=history
    )
    reply = response.choices[0].message.content
    history.append({"role": "assistant", "content": reply})
    return reply


def stream_message(client: OpenAI, history: list[dict], user_input: str) -> str:
```

```
    # why: the one function every later document/project calls to send a
    # message and get a streamed reply — no one else touches the SDK directly.
    history.append({"role": "user", "content": user_input})
    stream = client.chat.completions.create(
        model="gpt-4o-mini", messages=history, stream=True
    )
    full_reply = ""
    for chunk in stream:
        # how: each chunk carries one small piece of the reply; None means
        # this particular chunk carried no new text.
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="")
            full_reply += piece
    print()
    history.append({"role": "assistant", "content": full_reply})
    return full_reply


def get_structured_reply(
    client: OpenAI, user_input: str
) -> ExtractedInfo | None:
    # why: pulled out on its own so main.py's /extract command, and the
    # test file, can call it directly without going through the input() loop.
    try:
        completion = client.beta.chat.completions.parse(
            model="gpt-4o-mini",
            messages=[
                {"role": "system", "content": EXTRACT_PROMPT},
                {"role": "user", "content": user_input},
            ],
            response_format=ExtractedInfo,
        )
    except ValidationError:
        # when: the reply didn't fit the form — the caller gets None
        return None

    message = completion.choices[0].message
    # why: check refusal before parsed — a refusal leaves parsed as None
    if message.refusal:
        return None
    info = message.parsed
    # when: no name and no request — "nothing found", not an empty object
    if info.name is None and info.intent is None:
        return None
    return info
```

Story — `schemas.py`: without a schema, "give me structured data" is just a hope that the model's text happens to look like JSON. `ExtractedInfo` makes it a real, checked guarantee — either you get a valid object back, or a `ValidationError` you can catch. **If not:** `main.py` would have to hand-parse the model's raw text and hope it's valid JSON every time, silently breaking the moment the model phrases something slightly differently.

```
# schemas.py
from pydantic import BaseModel


class ExtractedInfo(BaseModel):
    # why: str | None, not str — a required field would force the model
    # to invent a name or a request when the message has none
    name: str | None
    intent: str | None
```

Story — `main.py`: this is where every practice exercise above actually comes together — config, logging, memory, streaming, structured output, and all 3 required failure cases, in one real terminal app. **If not:** you'd have 5 separate, disconnected scripts that each prove one thing works, but no single place proving they all work *together* — which is exactly what Project 1 and every later document assume this Build Task already did.

```
# main.py
import sys
import openai
from config import load_config
from logging_setup import get_logger
from chat_client import create_client, get_structured_reply, stream_message


def main() -> None:
    # why: loading config/logger first means a missing OPENAI_API_KEY fails
    # loudly here, at startup, before the terminal loop even opens.
    config = load_config()
    logger = get_logger(__name__)
    client = create_client()
    system_message = {
        "role": "system", "content": "You are a helpful assistant."
    }
    history: list[dict] = [system_message]

    while True:
        user_input = input("You: ")
        if user_input == "/quit":
            break
```

```
        try:
            if user_input.startswith("/extract "):
                # when: the user asked for the structured-output feature,
                # not a normal chat turn — a different code path entirely.
                text = user_input.removeprefix("/extract ")
                result = get_structured_reply(client, text)
                if result is None:
                    print("Couldn't understand that clearly enough to extract.")
                else:
                    print(result)
            else:
                stream_message(client, history, user_input)
        except openai.AuthenticationError:
            # how: a bad key is not something a retry ever fixes, so this
            # logs it, tells the user plainly, and exits — no scary traceback.
            logger.error("Authentication failed — check OPENAI_API_KEY.")
            print("Login failed — check your API key.")
            sys.exit(1)
        except openai.BadRequestError:
            # how: trimming to [system message] + last 4 turns keeps the
            # conversation alive instead of crashing — the model "forgets"
            # the oldest turns, a real trade-off, not a free fix.
            logger.warning("Conversation too long, trimming history.")
            print("That got too long — forgetting the oldest messages.")
            history = [history[0]] + history[-4:]


if __name__ == "__main__":
    main()
```

Expected output: the same streaming chat as Basic, plus typing `/extract Maria wants to reset her password` prints `name='Maria' intent='reset her password'`, and `/extract the sky is blue` prints the clean fallback message.

Story — `test_chat_client.py`: `chat_client.py` is about to get imported by Project 1, and rewritten (not from scratch) by Doc05 and Doc06 — a silent regression here (say, `get_structured_reply()` no longer checking `refusal`) would show up as a confusing bug in one of those, far from its real cause. This script runs all 5 Test Cases from the README in one go, right after writing the code. **If not:** you'd only find out something broke here whenever a future document happened to trigger it — much harder to trace back to this file.

```
# test_chat_client.py
# Runs the 5 Test Cases from the README against chat_client.py.
# Run it from inside this folder: python test_chat_client.py
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
```

```
from openai import OpenAI

from chat_client import create_client, get_structured_reply, send_message


def test_normal_conversation() -> None:
    # case 1: a normal 3-turn conversation should remember earlier turns
    client = create_client()
    history = [{"role": "system", "content": "You are a helpful assistant."}]
    send_message(client, history, "My favorite color is teal.")
    reply = send_message(client, history, "What's my favorite color?")
    print(f"case 1 (3-turn memory)        -> {reply}")


def test_bad_key() -> None:
    # case 2: a wrong API key should raise AuthenticationError, caught cleanly
    import openai

    bad_client = OpenAI(api_key="sk-fake-key-123")
    try:
        send_message(bad_client, [], "hi")
    except openai.AuthenticationError:
        print("case 2 (bad key)              -> AuthenticationError caught")


def test_context_limit() -> None:
    # case 3: a huge input should raise BadRequestError, not crash unhandled
    import openai

    client = create_client()
    huge_input = "word " * 200_000
    try:
        send_message(client, [], huge_input)
    except openai.BadRequestError:
        print("case 3 (context limit)        -> BadRequestError caught")


def test_structured_clear() -> None:
    # case 4: a clear input should come back as a valid ExtractedInfo
    client = create_client()
    result = get_structured_reply(client, "Maria wants to reset her password")
    print(f"case 4 (clear structured)     -> {result}")


def test_structured_unclear() -> None:
    # case 5: an unclear input should not silently produce a wrong answer
    client = create_client()
```

```
    result = get_structured_reply(client, "the sky is blue")
    print(f"case 5 (unclear structured)   -> {result}")


if __name__ == "__main__":
    test_normal_conversation()
    test_bad_key()
    test_context_limit()
    test_structured_clear()
    test_structured_unclear()
```

Expected output (wording/values vary run to run, since these are real calls):

```
case 1 (3-turn memory)        -> You mentioned your favorite color is teal!
case 2 (bad key)              -> AuthenticationError caught
case 3 (context limit)        -> BadRequestError caught
case 4 (clear structured)     -> name='Maria' intent='reset her password'
case 5 (unclear structured)   -> None
```

Case 5's `None` is expected, not a bug — "the sky is blue" has no name and no request, so the model fills both fields with `None` (the prompt and `str | None` allow that), and `get_structured_reply()` reports "nothing found" as `None`.

Difference from Basic: full type hints throughout, a real `schemas.py` module, `send_message()` and `get_structured_reply()` pulled out as their own reusable, testable functions instead of logic inlined in `main()`, and a `test_chat_client.py` that actually proves all 5 Test Cases pass — none of which Basic's version has yet.

Which one should you actually write? Intermediate's version already satisfies every Build Task requirement — full type hints, real memory, streaming, structured output, all 3 graceful-failure cases, and a test file that proves it. It's genuinely fine to ship as-is. A cached, explicitly-configured client (`OpenAI(max_retries=2, timeout=30.0)`, built once and reused) is worth adding once this code is meant to be imported and reused as a starting point for later projects — which is exactly what happens per this document's Goal — but it's a refinement, not a requirement.